DAY 5 · MODULE 5 · 4 HOURS

Agentic Batch Processing Implementation

Day 4 designed the four-zone pipeline on paper. Today you put code on it. Cascade-authored validator with the two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) added to .windsurfrules → v2. ASL-direct main pipeline (chunker → validator → lander), standalone SQS partial-batch demo, and a reusable DLQ replayer state machine — all riding on the design you ratified yesterday.

Accelya Kale · Agentic Development using AWS · Trainer: Jai Singh

RECAP · WHAT YOU SHIPPED YESTERDAY

Day 4 → Day 5

Yesterday you ratified the design. Today four Lambdas implement it — and Cascade rides the new pair of batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) added to .windsurfrules → v2 to scaffold each one against your design doc.

YESTERDAY

design/m4-batch-pipeline-design.md (ratified)

↓ becomes ↓

TODAY

Four Lambdas implementing the design: chunker (raw S3 → chunk payload), validator (row processing), lander (chunk-aggregate → manifest), dlq_replayer (reusable, env-driven). All ASL-direct in main pipeline; SQS only in Lab 2 demo.

YESTERDAY

Idempotency: 4 guards designed (row, file_sha256, canonical, business)

↓ provisioned as ↓

TODAY

ONE DynamoDB table idempotency_keys_<learner> with row#/file#/canonical#/business# PK prefixes (D4 contract). row# claimed in validator; file#/canonical# in lander.

YESTERDAY

.windsurfrules v1 (REPO RULES + CONSTRAINTS + EXAMPLES + REVIEW)

↓ extends to ↓

TODAY

v2 adds two blocks: ASL CHUNK CONTEXT (Lab 1 main pipeline) + SQS PARTIAL-BATCH CONTEXT (Lab 2 test queue demo).

YESTERDAY

5-section prompt anatomy + .windsurfrules v1 finalised

↓ becomes ↓

TODAY

5-section prompt unchanged. .windsurfrules grows two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT, both between CONSTRAINTS and EXAMPLES) → v2.

YESTERDAY

Failure-mode catalogue (6 categories + named catch-points)

↓ enforced by ↓

TODAY

Custom exceptions: ValidationError, ManifestMismatchError, MalformedRowError. ASL Catch routes them to quarantine.

YESTERDAY

5-dim HITL meta-checklist

↓ applied to ↓

TODAY

ASL JSON + Lambda code + DLQ replayer logic. Every artefact gets HITL_REVIEW.md.

From paper design to four working Lambdas — without losing the design

THE CHALLENGE — DESIGN-FAITHFUL CODE

Cascade can scaffold a Lambda in seconds. The hard part: making sure that Lambda honours the zone contracts, idempotency strategy, and failure-mode catch-points you committed yesterday.

Without discipline, the gap between design and code grows: validator hardcodes a path; DLQ becomes a graveyard; partial-batch failures get swallowed. By Day 7 you have working code that doesn't match the design anyone else can read.

The two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) added to .windsurfrules → v2 keep the gap small.

THE PAYOFF — SHIPPABLE PIPELINE BY EOD

By 03:30 today you'll have a 4-Lambda pipeline (chunker + validator + lander + dlq_replayer) live in your sandbox, processing real (synthetic) settlement data end-to-end: raw → validated, with quarantine working, DLQ feeding a replay tool that classifies failures.

Two state machines today: the ASL-direct main pipeline (chunker → validator → lander) and the reusable DLQ replayer (env-driven; demoed against Lab 2's standalone test queue). Plus a standalone partial-batch SQS demo that you bolt onto pipelines whenever bursty arrival demands it.

Tomorrow Day 6 takes that validated/ output (one settlement object per line — JSONL under s3://.../validated/<learner>/...) and teaches it to be queryable — Glue Crawler, Parquet, the CATALOG CONTEXT block in .windsurfrules (v3).

BY THE END OF TODAY YOU WILL BE ABLE TO

By the end of Day 5 you can…

AUTHOR

Cascade-author 4 Lambdas (chunker, validator, lander, dlq_replayer) using the 5-section prompt + .windsurfrules v2 with BATCH CONTEXT — chunk payload contract, idempotency contract, error routing.

ORCHESTRATE

Build the Step Functions state machine: Chunk (Task) → HasRows (Choice) → Validate (Map over chunk_keys) → Land (Task) + Pass NoRows. ASL invokes each Lambda directly with small reference payloads — NO SQS ESM in the main pipeline; chunk payloads stay in S3 (Step Functions state limit is 256 KiB).

PROVISION

Create the single DynamoDB idempotency table from the Day-4 design (idempotency_keys_<learner> with row#/file#/canonical#/business# PK prefixes — D4 contract; no learner choice).

PARTIAL-BATCH (separate teaching demo)

On a standalone test queue (not the main pipeline), wire SQS→Lambda ESM with ReportBatchItemFailures + BatchSize 10 + visibility = 6× test-Lambda timeout + batching window (Lab 2 uses 65s = 6×10s + 5s); prove row-failure vs message-failure semantics. Production-hardening pattern, not part of the D5 pipeline.

REPLAY

Build a DLQ replayer with 3 classifiers (transient → re-drive; data → quarantine; config → escalate) fed by the Lab-2 test queue's DLQ. Demonstrate one DLQ→replay→success cycle live.

REVIEW

Apply the 5-dim HITL meta-checklist to ASL JSON (not just code). Catch missing Retry, blanket Catch, missing ResultPath, hardcoded ARNs.

AGENDA · 4 HOURS

Four parts. Concepts → Design → Labs → Wrap.

01 Concept briefing

Seven concepts. Records[] reality. SQS as buffer. Step Functions basics. 3 functional Lambda stages: Chunk, Validate via Map, Land + 2 control states (HasRows/NoRows). DLQ as triage queue. .windsurfrules v2 with two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT). HITL on ASL.

TIME

~30 min

02 Guided design

Four guided slides. ASL anatomy. Idempotency across stages. Custom exceptions → ASL Retry/Catch routing. DLQ replayer state machine.

TIME

~35 min

03 Three labs

Lab 1 — Cascade-author the 4-Lambda pipeline (chunker + validator + lander + dlq_replayer) + ASL via 5-section prompt + .windsurfrules v2 (45 min). Lab 2 — provision standalone SQS test queue + DLQ + test-validator; demonstrate partial-batch live (45 min). Lab 3 — Build the DLQ replayer with classifiers (45 min).

TIME

~135 min

04 Review & wrap

Five Day-5 mistakes. Cleanup (now multi-Lambda). Day-5 in numbers. Bridge to Day 6 — Glue Crawler + Parquet + the CATALOG CONTEXT block in .windsurfrules (v3).

TIME

~25 min

01

PART 1 · ~30 MINUTES

Concept briefing

Seven concepts that make the difference between code that 'runs' and code that honours the design. Records[] reality. SQS as buffer (visibility math). Step Functions ASL primitives. 3 functional Lambda stages: Chunk, Validate via Map, Land + 2 control states (HasRows/NoRows). DLQ as triage queue (not graveyard). The two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) in .windsurfrules v2. HITL applied to ASL JSON.

CONCEPT 1 · RECORDS[] REALITY (production-hardening primer; demoed in Lab 2)

When you DO put SQS in front of a Lambda, your code must be batch-aware.

The D5 main pipeline (Lab 1) uses Step Functions to invoke each Lambda directly with the chunk payload — no SQS, no Records[]. But you'll bolt SQS+ESM onto pipelines whenever bursty arrival or fan-out demands it. Lab 2 demos this on a standalone test queue.

When SQS triggers a Lambda via ESM, the event is a list — `Records[]` of up to 10 messages by default. If your handler treats the batch as a single thing and one row throws, the WHOLE batch retries. 1 bad row kills 9 good ones. The fix is partial batch responses.

Batch-aware handler skeleton (annotated; used by Lab 2's test-validator)

```
def handler(event, context):
    failures = []
    for record in event['Records']:
        msg_id = record['messageId']  # SQS message ID, NOT row ID
        try:
            payload = json.loads(record['body'])
            file_key = payload['file_key']
            line_no  = payload['line_no']
            row      = payload['row']

            # 1. Row-replay idempotency claim (D4 prefixed PK contract)
            if not idempotency.claim(f'row#{file_key}:{line_no}', ttl_days=30):
                continue   # duplicate — silent skip; NOT a failure

            # 2. Schema validation
            try:
                schema.validate(row)
            except SchemaError as e:
                quarantine.put(file_key, line_no, row,
                               reason='schema_drift', detail=str(e))
                continue   # quarantined — NOT a failure for the batch

            # 3. Business-key idempotency claim before persist (defence in depth)
            settlement_id = row['settlement_id']
            if not idempotency.claim(f'business#{settlement_id}', ttl_days=30):
                continue   # business duplicate — silent skip

            # 4. Persist to validated/
            validated.put(file_key, line_no, row)
```

01 Records[] is the input

Always iterate. Never assume single-record.

02 messageId, not row_id

ReportBatchItemFailures uses SQS msg_id. Your row keys are separate.

03 Quarantine ≠ failure

Schema fails go to quarantine/. Don't add to batchItemFailures.

04 Duplicate ≠ failure

Idempotency hit returns silently. The message is acknowledged.

05 Real failures retry

Network blips, throttles, unhandled exceptions → batchItemFailures → SQS retries that message only.

Visibility timeout math, batch size, DLQ contract.

In production, SQS often sits between event producers and validators — buffering spikes, retrying failures, and routing poison messages to a DLQ. In Day 5, this pattern is demonstrated separately in Lab 2 (the main pipeline is ASL-direct). The same three knobs apply when you do reach for SQS: VisibilityTimeout, BatchSize, MaxReceiveCount.

Queue config that survives Day 5 lab traffic

```
# Validator queue (main work queue)
QueueName:               validator-queue-<learner>
VisibilityTimeout:       365 seconds   # 6 × 60s Lambda + 5s batching window — AWS guidance
MessageRetention:        1209600       # 14 days max
ReceiveMessageWaitTime:  20            # long-polling, free
RedrivePolicy:           {
  deadLetterTargetArn: arn:...validator-dlq-<learner>,
  maxReceiveCount:     3                 # 3 tries then DLQ
}

# Event Source Mapping (SQS → Lambda)
BatchSize:                      10       # lab default; SQS standard ESM supports up to 10000
(FIFO max 10)
MaximumBatchingWindowInSeconds:  5       # reduces invocations
FunctionResponseTypes:          [ReportBatchItemFailures]   # critical
ScalingConfig:                  { MaximumConcurrency: 10 }   # match Lambda cap

# Validator DLQ (poison-message triage)
QueueName:               validator-dlq-<learner>
VisibilityTimeout:       30 seconds
MessageRetention:        1209600          # full 14d for forensics
```

01 **VisibilityTimeout = 6×Lambda + batching window**

60s Lambda + 5s batching window → 365s visibility (AWS guidance). Too short → duplicate processing.

02 **BatchSize 10 (lab default)**

Standard SQS Lambda ESM supports up to 10000; FIFO max 10. We use 10 for predictable lab timing.

03 **ReportBatchItemFailures**

Function response type. Without it, partial-batch is dead.

04 **MaxReceiveCount 3**

3 tries then DLQ. Lower for cheap retries; higher for transient-prone work.

05 **ESM concurrency ≠ Lambda reserved**

ScalingConfig.MaximumConcurrency limits queue poller concurrency only — does NOT replace Lambda reserved concurrency controls.

ASL primitives — Task, Choice, Pass, Wait, Parallel — and why ASL is reviewable.

A Step Functions state machine is JSON (Amazon States Language). Each state has a Type, optional Retry/Catch arrays, and either Next or End. Five state types cover ~95% of pipelines.

validator-pipeline.asl.json — annotated pseudo-ASL (3 Lambda Tasks + 2 control states; named Catch routes). Copy-paste warning: this slide includes // comments for teaching; the real validator-pipeline.asl.json file must be comment-free JSON (strip // lines before aws stepfunctions validate-state-machine-definition).

```
{
  "Comment": "Validator pipeline: chunk → validate → land. ASL invokes Lambdas DIRECTLY (no SQS
ESM).",
  "StartAt": "Chunk",
  "States": {
    "Chunk": {
      "Type": "Task", "Resource": "arn:aws:states:::lambda:invoke",
      "Parameters": {"FunctionName":"accelya-chunker-${LEARNER}", "Payload.$":"$"},
      "OutputPath": "$.Payload",
      "Retry": [{"ErrorEquals":["Lambda.TooManyRequestsException","States.Timeout"],
                 "MaxAttempts":3, "IntervalSeconds":2, "BackoffRate":2.0}],
      "Catch": [{"ErrorEquals":["MalformedRowError"],
                 "ResultPath":"$.error", "Next":"QuarantineMalformed"},
                {"ErrorEquals":["ConfigError"],
                 "ResultPath":"$.error", "Next":"NotifyOps"}],
      "Next": "HasRows"
    },
    "HasRows": {
      "Type": "Choice",
      "Choices": [{"Variable":"$.row_count", "NumericGreaterThan":0, "Next":"Validate"}],
      "Default": "NoRows"
    },
    "Validate": {"Type":"Map", "ItemsPath":"$.chunk_keys", "MaxConcurrency":4,
                 "ResultPath":"$.validation_results",   // preserve
file_key/file_sha256/chunk_keys for Land
                 "ItemSelector":{                          // build each Map item with parent
context
                   "chunk_key.$":"$$.Map.Item.Value",
                   "file_key.$":"$.file_key",
                   "file_sha256.$":"$.file_sha256"
                 },
                 "Iterator":{"StartAt":"ValidateChunk", "States":{"ValidateChunk":{
                   "Type":"Task", "Resource":"arn:aws:states:::lambda:invoke",
```

01 Task

Calls a Lambda or SDK action. Most state machines are 80% Task states.

02 Choice

Branches on input. The state machine equivalent of an if statement.

03 Pass

Reshapes input/output without invoking anything. Plumbing.

04 Retry / Catch

Per-Task. Retry handles transients; Catch routes errors to a recovery state.

05 OutputPath: $.Payload

Without this, every Task wraps output in metadata. Set it explicitly.

Chunk → Validate → Land. Plus a Choice and a Pass. Why this exact shape.

The pipeline could be one Lambda. We split into three Lambda Tasks because each has a single responsibility — and each can fail / retry / branch independently in the state machine. Add the Choice + Pass and the ASL has 5 states total (3 Tasks + 2 control).

Why three Lambda Tasks (not one) — separation of concerns

ARCHITECTURE NOTE: ASL invokes each Lambda DIRECTLY with the chunk payload. NO SQS event-source mapping in this pipeline. Step Functions handles retry, fan-out and orchestration. The SQS+ESM+partial-batch pattern is taught separately in Lab 2 as a production-hardening technique.

CHUNK (Lambda Task) — reads the file from raw/, splits into N row-chunks, WRITES each chunk to s3://.../staging/<learner>/<file_sha256>/chunk-<NNNN>.jsonl, returns {file_key, file_sha256, row_count, chunk_keys: [s3_key, ...]} — small payload, no rows[] in the ASL state (Step Functions state payloads are limited to 256 KiB). Stateful by file_key only.

VALIDATE (Lambda Task) — invoked by ASL Map state over chunk_keys. Receives one chunk_key, READS the chunk JSONL from s3://.../staging/, iterates rows; for each: claim 'row#<file_key>:<line_no>' (skip if duplicate), schema-check, claim 'business#<settlement_id>' before persist; pass-rows to validated/, fail-rows to quarantine/<reason>/. Returns {validated, quarantined, duplicate, chunk_key, file_key, file_sha256}.

LAND (Lambda Task) — finalises the run: claims 'file#<file_sha256>' + 'canonical#<canonical_manifest_hash>' in idempotency_keys_<learner>, writes a per-file manifest object to s3://.../validated/<learner>/_manifests/<file>.json.

HasRows (Choice) — short-circuits empty inputs; saves Lambda cold-starts on no-op runs.

NoRows (Pass) — returns a clean 'empty input' status. ONLY for genuinely empty input — named Catch routes (below) handle real failures, not this Pass.

By Day 6 you'll append a CATALOG state (Glue Crawler invoke). The 3-Task base means extending the ASL is appending one state, not rewriting the machine.

01 — Single responsibility

Chunk doesn't validate. Validate doesn't catalog. Land doesn't decide.

02 — Independent retry

Validate retries 3× on Throttle; Land retries 1× on Conflict.

03 — Choice gates

HasRows skips empty inputs (Pass to NoRows). Real failures go through named Catch routes per Task — there is no HasFailedRows state.

04 — Pass for plumbing

Reshape outputs cheaply. Use it more than you think.

05 — Extensible

Day 6 adds CATALOG; Day 7 adds ENRICH. Same machine.

CONCEPT 5 · DLQ AS TRIAGE QUEUE

Not a graveyard. A queue with a triage worker behind it.

Most teams treat the DLQ as 'where messages go to die'. That's a leak. Real-world DLQs need a replayer with classifiers — transient (re-drive), data (quarantine), config (escalate).

DLQ replayer with 3 classifiers (Lab 3 deliverable)

```
def replayer_handler(event, context):
    """Triggered manually or on a schedule. Reads validator-dlq, classifies,
       routes. Hard cap: process 100 messages per invocation (lab-safe).
    """
    sqs = boto3.client('sqs', region_name='ap-south-1')
    DLQ = os.environ['DLQ_URL']
    MAIN = os.environ['MAIN_QUEUE_URL']

    MAX_REPLAY_ATTEMPTS = 3
    for msg in receive_up_to_100(sqs, DLQ):
        cls = classify(msg)   # → 'transient' | 'data' | 'config'
        attempts = int(msg.get('MessageAttributes', {})
                          .get('replay_attempt', {}).get('StringValue', '0'))
        if cls == 'transient' and attempts < MAX_REPLAY_ATTEMPTS:
            sqs.send_message(QueueUrl=MAIN, MessageBody=msg['Body'],
                MessageAttributes={
                    'replay_attempt':     {'DataType':'Number',
                                           'StringValue': str(attempts + 1)},
                    'original_message_id':{'DataType':'String',
                                           'StringValue': msg['MessageId']},
                    'last_error_type':    {'DataType':'String',
                                           'StringValue': last_error_type(msg)},
                    'first_seen_at':      {'DataType':'Number',
                                           'StringValue': str(first_seen_epoch(msg))}
                })
            sqs.delete_message(QueueUrl=DLQ, ReceiptHandle=msg['ReceiptHandle'])
        elif cls == 'transient':   # exceeded cap → escalate
            escalate_to_sns(msg, reason='transient_replay_cap_exceeded')
        elif cls == 'data':
            quarantine_persist(msg, reason='dlq_data_error')
            sqs.delete_message(QueueUrl=DLQ, ReceiptHandle=msg['ReceiptHandle'])
        else:  # config
            escalate_to_sns(msg)   # leave on DLQ for human review
```

01 **Transient → re-drive**

ThrottlingException / TimeoutError → push back to main queue.

02 **Data → quarantine**

SchemaError / MalformedRow → preserve in quarantine/dlq_data/.

03 **Config → escalate**

Unknown / IAM / wiring → SNS topic, leave on DLQ for human.

04 **Manual or scheduled**

Lab: invoke manually. Production: EventBridge every 15 min.

05 **Hard cap per invocation**

Bound the blast radius. Lab cap 100; production tunable.

Two distinct contracts: ASL CHUNK CONTEXT (Lab 1 main pipeline) + SQS PARTIAL-BATCH CONTEXT (Lab 2 demo).

Day 3 gave you the 5-section prompt + .windsurfrules v1. Day 4 finalised v1 with project values. Day 5 adds TWO related blocks to .windsurfrules — promoting it to v2. They cover the two batch shapes you'll write code against: ASL-direct chunk payloads (the main pipeline) and SQS Records[] (the production-hardening pattern, demoed standalone in Lab 2). Cascade picks the right one based on which Lambda it's scaffolding.

Two blocks to add to .windsurfrules (both between CONSTRAINTS and EXAMPLES)

```
## ASL CHUNK CONTEXT  (Lab 1 main pipeline; ASL invokes Lambdas DIRECTLY)
Input shape:
  event is the chunk payload from the prior ASL state, e.g.:
  { file_key, file_sha256, row_count, chunk_keys: ["staging/<learner>/<file_sha256>/chunk-
0001.jsonl", ...] }   # NO rows[] in ASL state — chunks live in S3
Chunk contract:
  Validator receives one chunk_key; reads JSONL from S3; iterates rows from that chunk. Per row:
    1. claim 'row#<file_key>:<line_no>' (ttl 30d). Conditional fail → silent skip.
    2. Schema-validate the row.
       SchemaError → quarantine/<reason>/ (counted; NOT raised).
    3. claim 'business#<settlement_id>' (ttl 30d). Conditional fail → silent skip.
    4. Persist to validated/<learner>/.
  Return {validated, quarantined, duplicate, file_key, file_sha256}.
Whole-chunk verdicts (raised → ASL Catch routes them):
  ValidationError       → ASL Catch → QuarantineSchemaDrift (>50% of rows fail)
  ManifestMismatchError → ASL Catch → QuarantinePartial
  MalformedRowError     → ASL Catch → QuarantineMalformed (chunker only)
  ConfigError           → ASL Catch → NotifyOps

## SQS PARTIAL-BATCH CONTEXT  (Lab 2 standalone test queue; ESM-triggered)
Input shape:
  event['Records'] = list of SQS messages (BatchSize 10 default).
  Each record.body = JSON {file_key, line_no, row, mode?}.
  record.messageId = SQS msg ID; use it for batchItemFailures.
Batch contract:
  Iterate Records[]. Per record:
    Same 4-step pattern as ASL CHUNK CONTEXT (idempotency → schema → business → persist).
  Row-level outcomes:
    SchemaError / business-dup → quarantine + ACK msg (NOT batchItemFailures).
```

01 — Two blocks, one rule

Same idempotency contract, different input shape: chunk payload vs Records[].

02 — Quarantine ≠ failure

Both blocks: schema fail = quarantine + count, NOT raise (ASL) or batchItemFailures (SQS).

03 — Cascade picks the right shape

Chunker/validator/lander → ASL CHUNK. Lab 2 test-validator → SQS PARTIAL-BATCH.

04 — Named exceptions matter

ASL Catch routes derive from class names; the v2 EXAMPLES section enumerates all 5.

05 — Carries forward

Day 6 keeps both blocks; Day 7 adds STREAM CONTEXT (Kinesis Records[]).

The 5 dimensions apply to ASL JSON. Yes, JSON.

ASL is reviewable code. The 5-dim HITL meta-checklist (Correctness, Security, Observability, Scalability, Cost — D3 contract) catches mistakes ASL files make that ordinary linters miss.

HITL_REVIEW.md applied to validator-pipeline.asl.json

CORRECTNESS — does the state machine match the design? Each Day-4 failure mode has a named ASL state or Catch route?

SECURITY — IAM execution role scoped to required Lambda ARNs only? No `arn:aws:lambda:*:*:function:*` wildcards in `Resource`.

IDEMPOTENCY — does each Task's input contain enough to be replayed safely? OutputPath: $.Payload set on every Lambda Task to avoid double-wrapping?

OBSERVABILITY — every Task has Retry with named ErrorEquals AND a Catch with ResultPath: $.error so the error is preserved in the input for the next state?

COST — Choice states short-circuit empty inputs (no wasted Lambda invocations)? No Wait states with hardcoded long durations? No infinite-loop risk in Catch routing?

01

BOTH per Lambda Task

OutputPath:$.Payload AND Catch[].ResultPath:$.error. The two together. One without the other breaks downstream.

02

Retry with named errors

ErrorEquals must be specific. States.ALL is a smell.

03

OutputPath: $.Payload

Optimised Lambda Task wraps output. Strip metadata unless next state intentionally consumes it.

04

No infinite Catch loops

Catch → recovery state must End or call a distinct Task.

05

IAM least-privilege

States execution role lists every Lambda ARN explicitly.

PART 1 WRAP-UP

Concepts complete — implementation vocabulary set

Records[]. SQS contract. ASL primitives. 3 functional Lambda stages (Chunk → Validate via Map → Land). DLQ as triage. .windsurfrules v2 with two batch blocks. HITL on ASL. Now: guided design — translate these concepts into the artefacts the labs will build.

VOCABULARY

Records[], batchItemFailures, ASL Task/Choice/Pass, Catch with ResultPath, .windsurfrules v2 with two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) — every Day-5+ Cascade prompt uses these terms.

WHAT'S NEW

.windsurfrules v2 with two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT). ASL JSON joins the artefact set. DLQ replayer pattern. Custom exception → ASL Catch routing as the discipline.

WHAT'S NEXT

Four guided design slides: ASL anatomy with 3 Lambda Tasks + 2 control states; idempotency across stages; custom exception ladder; DLQ replayer state machine.

02

PART 2 · ~35 MINUTES

Guided design

Three concrete artefacts you'll Cascade-author in the labs. The full ASL annotated by section. Idempotency contract per stage (validator vs lander). Custom exception ladder mapped to ASL Catch routes.

Annotated reference. Each section explains one ASL primitive and how it maps to a Day-5 deliverable.

RETRY POLICY (per Lambda Task)

Three retries with backoff for transient errors. Custom errors don't retry — they Catch.

```
"Retry": [
  { "ErrorEquals": ["Lambda.ServiceException",

"Lambda.AWSLambdaException",

"Lambda.SdkClientException",

"Lambda.TooManyRequestsException"],
    "IntervalSeconds": 2, "MaxAttempts": 3,
    "BackoffRate": 2.0 },
  { "ErrorEquals": ["States.Timeout"],
    "IntervalSeconds": 5, "MaxAttempts": 2,
    "BackoffRate": 2.0 }
]

# Notable: ValidationError,
ManifestMismatchError, MalformedRowError
# are NOT in Retry — they are caught and routed
(next column).
```

CATCH ROUTING (per Lambda Task)

Custom exceptions route to recovery states; ResultPath preserves the error.

```
"Catch": [
  { "ErrorEquals": ["ValidationError"],
    "ResultPath": "$.error",
    "Next": "QuarantineSchemaDrift" },
  { "ErrorEquals": ["ManifestMismatchError"],
    "ResultPath": "$.error",
    "Next": "QuarantinePartial" },
  { "ErrorEquals": ["MalformedRowError"],
    "ResultPath": "$.error",
    "Next": "QuarantineMalformed" }
]

# Each Quarantine* state is a Pass that LABELS
the failure (status only). The Lambda MUST
persist the
# quarantine artefact + sidecar JSON to
s3://.../quarantine/<reason>/<learner>/...
BEFORE raising the
# routed exception. Pass states do not move
data — they only tag the run outcome.
# ResultPath: $.error keeps the original input
intact for forensics.
```

INPUT/OUTPUT SHAPING

OutputPath: $.Payload strips Lambda wrapping. Pass states reshape between Tasks.

```
"Validate": {
  "Type": "Task",
  "Resource": "arn:aws:states:::lambda:invoke",
  "Parameters": {
    "FunctionName": "accelya-validator-
${LEARNER}",  # literal (raw ASL); use SAM
intrinsics later
    "Payload.$": "$"   # CRITICAL: pass the
WHOLE chunk payload through.
                        # Validator receives
event['chunk_key'] AND event['file_sha256'];
                        # reads the chunk
JSONL from S3, then iterates rows.
                        # cherry-picking
file_key/chunk_id/row_count would break it.
  },
  "OutputPath": "$.Payload",   # critical:
strip Lambda invoke wrapper
  "Next": "Land"
}

# Without Payload.$:'$' (or with a malformed
Map ItemSelector), validator throws
KeyError('chunk_key') — or loads no rows from
S3.
# Without OutputPath, $.Payload.file_sha256 is
buried under $.ExecutedVersion etc.,
# and Land's file# claim breaks.
```

Day 4 designed 4 guards. Day 5 implements them across two stages: the validator handles row-level; the lander handles file-level + canonical.

VALIDATOR — row# + business#

Per record. row# claim first. Schema-validate. THEN business# claim (settlement_id or ticket_number+bsp_cycle+agency_iata+amount). Silent

```
# In src/validator/handler.py — ASL-direct (NOT
SQS Records[])
def handler(event, context):
    file_key      = event['file_key']
    counts = {'validated': 0, 'quarantined': 0,
'duplicate': 0}
    chunk_key = event['chunk_key']    # ASL Map
item — chunk_key is the S3 key
    rows = read_jsonl_from_s3(chunk_key)  #
read the chunk JSONL we wrote in chunker
    for item in rows:                     #
iterate rows from this chunk
        line_no, row = item['line_no'],
item['row']

        # 1) row#  — same-file replay guard
        if not
idempotency.claim(f'row#{file_key}:{line_no}'):
            counts['duplicate'] += 1
            continue

        # 2) schema validation
        try:
            schema.validate(row)
        except SchemaError as e:
            quarantine.put(file_key, line_no,
row, reason='schema_drift', detail=str(e))
            counts['quarantined'] += 1
            continue
        # 3) business# — semantic dedup, BEFORE
persist
```

LANDER — file + canonical

Per chunk completion. Computes file_sha256 + canonical_hash from Day 4 design.

```
# In src/lander/handler.py — runs ONCE after
Validate Map completes
def land_run(event):
    file_key      = event['file_key']
    file_sha256   = event['file_sha256']
    chunk_keys    = event['chunk_keys']
    chunk_results = event['validation_results']
# Map output array

    # Aggregate per-chunk Map results
    counts = aggregate(chunk_results)
# {validated, quarantined, duplicate}

    # File-byte guard (whole-file claim, once
per run)
    if not
idempotency.claim(f'file#{file_sha256}',
ttl_days=90):    # 90-day audit window
        return {'status': 'duplicate_file',
'file_sha256': file_sha256}

    # Canonical guard (over aggregated
validated rows)
    rows = read_validated_rows(file_key)
    canonical_hash = canonicalize_hash(rows)
    if not
idempotency.claim(f'canonical#{canonical_hash}'
, ttl_days=90):    # 90-day audit window
        return {'status':
'duplicate_canonical'}

    write_manifest(file_key, file_sha256,
```

TABLE SCHEMA (consolidated)

One DynamoDB table; PK prefix routes the guard. Lab simplification from Day 4.

```
TABLE: idempotency_keys_<learner>
  pk            String   = '<guard>#<value>'

row#<file_key>:<line_no>

file#<file_sha256>

canonical#<canonical_hash>

business#<settlement_id>
  outcome       String
  processed_at  Number
  ttl           Number

TTL strategy:
  row#         → 30 days  (BSP cycle + dispute)
  file#        → 90 days  (longer audit window)
  canonical#   → 90 days
  business#    → 90 days

Splitting into 4 tables is a production option;
Day 5 lab uses one.
```

The exception ladder is the contract between Cascade-authored Lambdas and the ASL state machine. Name them now; Day 6+ inherits.

EXCEPTION LADDER (Python, src/common/exceptions.py)

Custom exception per failure mode. Inherits from a base for clean catch.

```
class PipelineError(Exception):
    """Base for all pipeline-routed errors."""
    def __init__(self, message, *,
file_key=None, line_no=None,
                 chunk_id=None, detail=None):
        super().__init__(message)
        self.file_key, self.line_no = file_key,
line_no
        self.chunk_id, self.detail  = chunk_id,
detail

class ValidationError(PipelineError):
"""Schema mismatch."""
class ManifestMismatchError(PipelineError):
"""Manifest row count mismatch."""
class MalformedRowError(PipelineError):
"""JSON parse failure."""
class TransientError(PipelineError):
"""Throttle / timeout / 5xx."""
class ConfigError(PipelineError):
"""IAM, ARN, env-var, or wiring."""
```

ASL CATCH MAP (per Task)

Each named exception has its own Catch with ResultPath: $.error.

```
// applied to Validate Task in validator-
pipeline.asl.json
"Catch": [
  { "ErrorEquals": ["ValidationError"],
    "ResultPath": "$.error", "Next":
"QuarantineSchemaDrift" },
  { "ErrorEquals": ["ManifestMismatchError"],
    "ResultPath": "$.error", "Next":
"QuarantinePartial" },
  { "ErrorEquals": ["MalformedRowError"],
    "ResultPath": "$.error", "Next":
"QuarantineMalformed" },
  { "ErrorEquals": ["ConfigError"],
    "ResultPath": "$.error", "Next":
"NotifyOps" }
]

# TRAINER NOTE: Lambda runtime can wrap
exceptions; the visible error
# name in Step Functions may not match the
Python class. In Lab 1,
# force one ValidationError and one
TransientError once and confirm the
# ErrorEquals string the state machine actually
sees before relying on it.
```

DLQ REPLAYER STATE MACHINE

Replayer is its own short ASL — read DLQ, classify, route. EventBridge schedule (or manual).

```
{
  "StartAt": "DrainDLQ",
  "States": {
    "DrainDLQ": {
      "Type": "Task",
      "Resource":
"arn:aws:states:::lambda:invoke",
      "Parameters": {
        "FunctionName": "accelya-dlq-replayer-
${LEARNER}",
        "Payload": { "max_messages": 100 }
      },
      "OutputPath": "$.Payload",
      "Next": "Classify"
    },
    "Classify": {
      "Type": "Choice",
      "Choices": [
        { "Variable": "$.transient_count",
"NumericGreaterThan": 0,
          "Next": "NotifyOps" }
      ],
      "Default": "Done"
    },
    "NotifyOps": { "Type":"Task",
"Resource":"arn:aws:states:::sns:publish",
                  "End":true },
    "Done": { "Type":"Pass", "End":true }
```

PART 2 WRAP-UP

Design ratified for implementation — three artefacts ready to build

ASL JSON, idempotency staging, exception ladder. The labs now turn each into committed code with HITL_REVIEW.md.

ASL FRAMEWORK

Retry + Catch + OutputPath + Choice gates — the 4 patterns every state machine in this programme uses.

IDEMPOTENCY STAGES

Validator owns row#; Lander owns file# + canonical#. One DynamoDB table; PK prefix routes the guard.

EXCEPTION CONTRACT

5 named exceptions. Each has its ASL Catch route. Cascade enforces from now on via .windsurfrules.

03

PART 3 · ~135 MINUTES

Three labs

Lab 1 (45 min) — Cascade-author chunker+validator+lander+ASL via 5-section prompt + .windsurfrules v2 (ASL CHUNK CONTEXT); provision DynamoDB only (no SQS in main pipeline). Lab 2 (45 min) — Wire the SQS event source mapping + DLQ; observe partial-batch failure live. Lab 3 (45 min) — Build the DLQ replayer with classifiers; demonstrate one DLQ→replay→success cycle.

LAB 1 · ~45 MINUTES

Promote .windsurfrules to v2 + Cascade-author all 3 ASL Lambdas + ASL state machine

BRIEF

Pair-work. Add the two batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) to .windsurfrules (between CONSTRAINTS and EXAMPLES) — this promotes it from v1-final (yesterday) to v2. Cascade then reads BATCH CONTEXT on every prompt and scaffolds: chunker + validator + lander handlers + tests + src/common/exceptions.py + infra/validator-pipeline.asl.json — using your standard 5-section prompt.

Provision the SINGLE DynamoDB table idempotency_keys_<learner> (D4 contract — row#/file#/canonical#/business# PK prefixes) via Cascade-authored CLI script. (SQS queues are NOT in the main pipeline — Lab 2 wires a separate test queue for the partial-batch demo.)

Deploy all THREE Lambdas (chunker, validator, lander) — the ASL state machine fails at runtime if any are missing.

HITL_REVIEW.md the ASL JSON specifically — most reviews focus on Python and skip ASL.

TIME

45 min

LEVEL

Working+

DELIVERABLE

.windsurfrules v2 (TWO blocks added: ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT; commit: 'promote v1-final → v2')
src/{chunker,validator,lander}/handler.py + tests + src/common/exceptions.py
infra/validator-pipeline.asl.json (3 Lambda Tasks + 2 control states)
scripts/provision_m5_resources.sh (DynamoDB only — no SQS in main pipeline; Lab 2 owns m5-test-queue + m5-test-dlq)
Three deployed Lambdas: accelya-chunker-<learner>, accelya-validator-<learner>, accelya-lander-<learner>
/reviews/HITL_REVIEW-m5-asl.md (ASL-specific review)

Six steps. Branch → .windsurfrules v2 → scaffold → provision → test → MR.

1

Branch + add TWO batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) to .windsurfrules

```
export LEARNER=jai
git checkout main && git pull --ff-only
git checkout -b module-5-$LEARNER
# Open .windsurfrules. Insert BOTH batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) between
```

EXPECTED

On module-5-<learner>. .windsurfrules v2 with BOTH batch blocks (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT) visible between CONSTRAINTS and EXAMPLES.

2

Cascade scaffolds chunker + validator + lander + exceptions

```
# In Windsurf chat (5-section prompt — your standard from Day 3):
# TASK: Scaffold THREE Lambda handlers + shared exceptions per ASL CHUNK CONTEXT.
#       Use ASL CHUNK CONTEXT only — NOT SQS PARTIAL-BATCH CONTEXT (that's Lab 2).
#       Reference design/m4-batch-pipeline-design.md
```

EXPECTED

Four src trees present. All three handlers receive payload directly from ASL. All reference the SINGLE idempotency_keys_<learner> table with prefixed PKs.

3

Cascade scaffolds ASL

```
# Same chat (continues context):
# TASK: Scaffold infra/validator-pipeline.asl.json with 3 Lambda Tasks +
#       2 control states per Concept 4. Include Retry + Catch with
#       ResultPath: $.error (preserves $.payload for all Lambda tasks
#   row_count, chunk_keys[], s3://staging/... — small payload (no rows[] in ASL state).
```

EXPECTED

ASL valid. Catch routes named for each PipelineError subclass.

4

Provision DynamoDB (NO SQS in main pipeline — Lab 2 provisions test queue)

```
bash scripts/provision_m5_resources.sh $LEARNER
# Creates: idempotency_keys_<learner> (DynamoDB, on-demand, TTL enabled on 'ttl')
# Verify table is ACTIVE:
# 3) src/lander/handler.py — ASL-invoked. Claims 'file#<file_sha256>' +
```

EXPECTED

Table ACTIVE. TTL TimeToLiveStatus=ENABLED on AttributeName='ttl'.

5

Package + deploy all 3 Lambdas + render & create state machine (ASL-direct; no SQS ESM)

```
# Package each handler:
for FN in chunker validator lander; do
  (cd src/$FN && zip -rq ../../$FN.zip .)
done
```

EXPECTED

All 3 Lambdas Active. State machine ARN captured in $SM_ARN. NO ESM yet (Lab 2 wires a separate test-queue ESM).

6

HITL review of ASL + commit + MR

```
# Score ASL on 5 dimensions (see Concept 7).
# Save to /reviews/HITL_REVIEW-m5-asl.md.
git add .windsurfrules src/ tests/ infra/ scripts/ reviews/
git commit -m "Module 5: .windsurfrules v2 + chunker/validator/lander + ASL (Lab 1)"
git push -u origin module-5-$LEARNER
# Open MR module-5-<learner> → module-5-base; tag pair partner
```

EXPECTED

MR open. ASL HITL review attached. Pair partner tagged.

Success criteria & common gotchas

✓ YOU ARE READY IF…

· src/{chunker,validator,lander}/handler.py all present with tests
· src/common/exceptions.py defines 5 PipelineError subclasses
· infra/validator-pipeline.asl.json validates: 3 Lambda Tasks + 2 control states + 4 named Catch routes (Quarantine* + NotifyOps); ASL invokes Lambdas DIRECTLY (no SQS in pipeline)
· DynamoDB idempotency_keys_<learner> ACTIVE; describe-time-to-live shows ENABLED on 'ttl'
· All 3 Lambdas (chunker, validator, lander) Active; state machine ARN captured. (NO SQS ESM yet — that's Lab 2.)
· HITL ASL review scored ≥ 12/15; MR open with pair partner tagged

⚠ TOP GOTCHAS

Skipping the right batch block

Lab 1 validator receives event['chunk_key'] from the ASL Map (rows are loaded from S3); Lab 2 validator receives event['Records'] from the SQS event-source mapping. Verify .windsurfrules v2 has BOTH blocks and Cascade picks the right one per task (lab verifies).

Missing ResultPath on Catch

Catch overwrites the input. Without ResultPath, recovery state has no context.

OutputPath omitted

$.Payload wraps in metadata. Land Task receives unparseable input.

Visibility timeout < 6× handler + batching window

Duplicate processing. AWS guidance: VisibilityTimeout = 6 × Lambda timeout + MaxBatchingWindow. Lab uses 60s Lambda + 5s window → set 365s.

Quarantine = batchItemFailures

Schema fails are NOT batch failures. Don't add them — SQS will retry forever.

LAB 2 · ~45 MINUTES

Standalone partial-batch demo (separate test queue + dedicated test-validator Lambda)

BRIEF

Solo. This lab is a SEPARATE teaching demo, not part of the main D5 pipeline. You provision a standalone test-queue + test-DLQ + a tiny test-validator Lambda wired via SQS ESM with ReportBatchItemFailures. The main pipeline (Lab 1) does NOT use SQS — that's intentional. SQS+ESM+partial-batch is a production-hardening pattern you'll bolt onto the pipeline when bursty arrival or fan-out demands it.

Provision: m5-test-queue + m5-test-dlq (VisibilityTimeout 65s = 6×10s + 5s window — production formula scaled to the test-validator's 10s timeout). Deploy a tiny m5-test-validator that treats body as {file_key, line_no, row, mode?} and raises TransientError on mode='force_transient' (no message-attribute mutation — Lambda can't do that). The DEMO PRODUCER sends the sentinel WITH the error_type MessageAttribute already attached; SQS preserves attributes through DLQ redrive.

Run TWO distinct demos that prove the row/message-failure separation:
· Demo A — 10-row test batch with 1 schema-fail row → row goes to quarantine/, SQS message ACKed, ZERO entries in batchItemFailures. test-DLQ stays empty.
· Demo B — single sentinel message with mode='force_transient' AND MessageAttribute error_type=TransientError → batchItemFailures = [{itemIdentifier: messageId}], SQS retries maxReceiveCount=3 times, then routes to test-DLQ (with the error_type attribute preserved).

TIME

45 min

LEVEL

Working+

DELIVERABLE

m5-test-queue + m5-test-dlq + m5-test-validator (TEMPORARILY ACTIVE during Lab 2/3 — count rises to 6 of 10 while alive; cleanup at end of D5 deletes test-validator + queues, returning final count to 5)
/reviews/m5-partial-batch-evidence.md — Logs Insights output + DLQ depth screenshot reference.

Six steps. Provision test-queue (lab-short visibility) → deploy test-validator → wire ESM → Demo A → Demo B → cleanup-ready.

1

Provision standalone m5-test-queue + m5-test-dlq (LAB-SHORT visibility from the start)

```
# IMPORTANT: this is a LAB-ONLY queue with VisibilityTimeout=65s = 6×10s test-validator + 5s window.
# (Production formula is identical: 6×Lambda + batching window.) Set up front to avoid races.
TEST_DLQ_URL=$(aws sqs create-queue --queue-name m5-test-dlq-$LEARNER \
```

EXPECTED

Both queue URLs captured into env vars. Test-queue has redrive to test-DLQ at maxReceiveCount=3.

2

Deploy m5-test-validator (lab-only; will be deleted at cleanup)

```
# Cascade-author src/m5_test_validator/handler.py — Use SQS PARTIAL-BATCH CONTEXT only.
#   - Records[] handler
#   - For each msg: body has {file_key, line_no, row, mode?}
```

EXPECTED

Lambda Active. Cumulative count temporarily 6 of 10 while m5-test-validator is alive (until cleanup).

3

Wire ESM with ReportBatchItemFailures (CRITICAL flag)

```
QUEUE_ARN=$(aws sqs get-queue-attributes --queue-url $TEST_QUEUE_URL \
  --attribute-names QueueArn --query 'Attributes.QueueArn' --output text --region ap-south-1)
```

EXPECTED

ESM ACTIVE on m5-test-validator with FunctionResponseTypes=[ReportBatchItemFailures].

4

Demo A — 10 rows + 1 schema-fail row → quarantine + ACK

```
python3 -m scripts.gen_test_batch \
  --rows 10 --poison-line 4 \
  --out /tmp/m5-test-batch-sqs.json
```

EXPECTED

9 rows event=row_landed. 1 row event=row_quarantined reason=schema_drift. Zero entries in batchItemFailures. Test-DLQ stays empty.

5

Demo B — sentinel mode='force_transient' WITH error_type MessageAttribute

```
# Send sentinel WITH the error_type attribute set explicitly (CRITICAL for Lab 3 classifier).
# When SQS DLQ-redrives this message, the MessageAttribute is preserved on the DLQ message,
# giving the dlq_replayer something to classify on.
```

EXPECTED

After ~240s, test-DLQ ApproximateNumberOfMessages = 1. The DLQ message's MessageAttributes show error_type=TransientError (preserved through the redrive). This is what Lab 3's classifier reads.

6

Document evidence + commit (cleanup deferred to end of D5)

```
# /reviews/m5-partial-batch-evidence.md captures:
#   - Demo A: 9 validated, 1 quarantined, 0 batchItemFailures (test-DLQ empty)
#   - Demo B: 1 sentinel → 3 retries → test-DLQ has 1 message with error_type attr
```

EXPECTED

MR shows commits. Evidence file shows Demo A (9-pass / 1-quarantine / 0-DLQ) AND Demo B (1 sentinel → 3 retries → 1-DLQ with error_type attribute preserved).

Success criteria & common gotchas

✓ YOU ARE READY IF…

· m5-test-queue + m5-test-dlq provisioned (VisibilityTimeout=65s = 6×10s test-validator + 5s window; URLs captured into env vars)
· m5-test-validator deployed with ESM (ReportBatchItemFailures + BatchSize=10) — separate from main pipeline
· Demo A — 10-row batch: 9 validated, 1 quarantined, ZERO entries in batchItemFailures, test-DLQ empty
· Demo B — sentinel sent WITH error_type=TransientError MessageAttribute; test-DLQ has 1 message after ~240s with attribute preserved
· /reviews/m5-partial-batch-evidence.md captures Logs Insights output for both demos
· MR shows 2 commits

⚠ TOP GOTCHAS

ReportBatchItemFailures off

Default behaviour: any error fails whole batch. ESM must enable the response type.

BatchSize too low to test

BatchSize=1 hides partial-batch behaviour. Use 10 to demonstrate.

Visibility too short (production)

<6×Lambda + batching window → duplicate processing during retry. Lab uses 65s because the test-validator timeout is 10s and batching window is 5s (6×10 + 5 = 65). Production: same formula scaled to your Lambda's timeout.

Quarantine added to failures

Most common Cascade bug. Schema fails are silent, not batch failures.

Forgetting to set scaling-config

ESM concurrency unbounded; you can overrun downstream capacity (DynamoDB throttling, S3 503s). ScalingConfig.MaximumConcurrency caps the queue poller — does NOT change the Lambda function count.

LAB 3 · ~45 MINUTES

Build a REUSABLE DLQ replayer with classifiers — drains Lab 2's m5-test-dlq today; repointable to any future DLQ

BRIEF

Pair-work. Cascade-author the DLQ replayer Lambda + the dlq-replayer state machine. Replayer is FULLY env-driven: DLQ_URL + MAIN_QUEUE_URL + ESCALATION_TOPIC are environment variables — same Lambda can drain any production DLQ later by repointing env. Today it points at Lab 2's m5-test-dlq for the cycle demo, but it's a programme Lambda (counts as 1 of 4 new today; stays after cleanup).

Three classifiers driven off the error_type MessageAttribute Lab 2's sentinel preserved on the DLQ message: transient (re-drive to MAIN_QUEUE_URL), data (move to quarantine/dlq_data/), config (escalate to ESCALATION_TOPIC SNS).

Demonstrate one full DLQ→replay→success cycle: take the DLQ sentinel from Lab 2's m5-test-dlq (error_type=TransientError; row payload includes a valid settlement_id), TOGGLE the test-validator's force_transient code path off (FORCE_TRANSIENT_ENABLED=false in env), re-drive via the replayer, observe the row land in validated/.

TIME

45 min

LEVEL

Working+

DELIVERABLE

src/dlq_replayer/handler.py + tests
infra/dlq-replayer.asl.json
/reviews/m5-dlq-replay-cycle.md (one DLQ→replay→success cycle documented; row appears in validated/)

Six steps. Cascade-author replayer → SNS topic → state machine → fix → replay → verify.

1 Cascade-author the replayer Lambda

```
# In Windsurf chat (continues from Lab 1):
# TASK: Scaffold src/dlq_replayer/handler.py per Concept 5.
# 3 classifiers + 100-message hard cap. EVENT: { max_messages: 100 }.
# Reads $DLQ_URL, classifies, routes. Returns { transient_count
```

EXPECTED

src/dlq_replayer/handler.py + tests/test_dlq_replayer.py present.

2 Provision SNS escalation topic

```
aws sns create-topic --name accelya-pipeline-escalations-$LEARNER \
  --region ap-south-1
ESCALATION_TOPIC=$(aws sns list-topics --region ap-south-1 \
```

EXPECTED

Topic created. Email subscription PendingConfirmation (confirm via inbox).

3 Deploy replayer + create dlq-replayer state machine

```
(cd src/dlq_replayer && zip -r ../../replayer.zip . && cd ../..)
aws lambda create-function \
  --function-name accelya-dlq-replayer-$LEARNER \
```

EXPECTED

New today: 4 Lambdas (chunker, validator, lander, dlq_replayer). Cumulative: 5 of 10 (including D2 starter validator). Replayer points at Lab 2's m5-test-dlq.

4 Fix the upstream condition (Lab-2 sentinel transient)

```
# Lab 2's DLQ message has mode='force_transient' AND error_type=TransientError attribute.
# The fix: toggle the test-validator's force_transient code path OFF.
aws lambda update-function-configuration \
```

EXPECTED

Test-validator env updated; FORCE_TRANSIENT_ENABLED=false. Replay will now process the sentinel as a normal row.

5 Run the replayer state machine

```
aws stepfunctions start-execution \
  --state-machine-arn $REPLAYER_SM_ARN \
  --input '{"max_messages":100}' \
# → row# + business# idempotency claims succeed → row written to validated/.
```

EXPECTED

test-DLQ depth 0. test-queue briefly +1 then drains. validated/ contains a row referencing settlement_id BSP-2026-W19-RECOVER-001.

6 Document cycle + final commit

```
# Capture the cycle in /reviews/m5-dlq-replay-cycle.md:
#   T0: 1 message in DLQ (from Lab 2)
#   T1: upstream fix applied
#   T2: replayer classifies as transient → re-drives to main queue
# Verify the row landed in validated/:
#   T3: validator processes; row appears in validated/
#   T4: DLQ depth = 0
```

EXPECTED

MR shows 3 commits. Replay cycle documented end-to-end.

Success criteria & common gotchas

✓ YOU ARE READY IF…

- src/dlq_replayer/handler.py with 3 classifiers + 100-msg cap
- infra/dlq-replayer.asl.json valid
- DLQ message from Lab 2 fixed and replayed → DLQ depth 0
- Validated/ contains the recovered row
- /reviews/m5-dlq-replay-cycle.md captures the full T0→T4 cycle
- MR shows 3 commits; pair partner tagged

⚠ TOP GOTCHAS

No 100-message cap

Replayer reads infinitely. Always bound the blast radius.

Config-class swallows transient

If classifier is too permissive, real bugs go silent. Default to 'config' = escalate, not silent.

Replayer re-drives faster than fix

If you replay before fixing the upstream, you'll loop. Always fix-first, replay-second.

No SNS subscription confirmed

Escalation goes to nowhere. Check email, click confirm before trusting.

Forgetting to delete from DLQ

Replayer must delete after re-drive or quarantine. Double-process risk otherwise.

Code shipped. Branches pushed. Time to step back.

04

PART 4 · ~25 MINUTES

Review & wrap

Five Day-5 mistakes that swallow cohorts. Cleanup that drains queues but preserves the state machine (Day 6 inherits). Day-5 in numbers. Bridge to Day 6 — Glue Crawler + Parquet + a CATALOG CONTEXT block in .windsurfrules (v3).

Five Day-5 mistakes — and what to do instead

01 Quarantine in batchItemFailures

MISTAKE Schema fails added to batchItemFailures. SQS retries the message; validator quarantines it again; loop.

DO INSTEAD Quarantine writes are silent successes. Only TRUE failures (transient/unhandled) belong in batchItemFailures.

02 Catch without ResultPath

MISTAKE Catch fires; recovery state receives only the error; original input is lost; can't quarantine properly.

DO INSTEAD Always set ResultPath: $.error. Original input stays at $.; error annotated at $.error.

03 Visibility timeout too short

MISTAKE VisibilityTimeout < 6× Lambda timeout. Lambda is mid-processing when the message becomes visible again; duplicate processing.

DO INSTEAD VisibilityTimeout ≥ 6× Lambda timeout (AWS guidance). For a 60s validator use 360s minimum.

04 Hardcoded ARNs in ASL

MISTAKE ASL JSON has 'Resource': 'arn:aws:lambda:ap-south-1:123456789012:function:validator'. Breaks on deploy to a new account.

DO INSTEAD Use ${AWS::AccountId}, ${AWS::Region}, or SAM intrinsic functions. Day 11 lifts to SAM template.

05 DLQ as graveyard

MISTAKE No replayer. DLQ accumulates; on-call manually re-drives; classification by gut feel.

DO INSTEAD DLQ replayer with 3 classifiers + 100-msg cap. EventBridge schedule for production; manual for lab.

End-of-module cleanup — preserve main pipeline; delete Lab 2/3 lab-only artefacts

CLEANUP CHECKLIST

- Branch pushed: module-5-<learner>
- MR open with 3 commits (Lab 1 scaffold, Lab 2 partial-batch evidence, Lab 3 replay cycle)
- Pair partner tagged on MR
- PRESERVE: validator-pipeline state machine + chunker/validator/lander Lambdas + DDB idempotency table + validated/quarantine S3 paths (Day 6 reads from these)
- DELETE: m5-test-validator (lab-only) + ESM + m5-test-queue + m5-test-dlq (after evidence captured) — returns Lambda count from temporary 6 → final 5 of 10
- DECISION (lab-default): KEEP dlq-replayer state machine + accelya-dlq-replayer Lambda — counted as a programme Lambda; env vars can be repointed at any future DLQ (production-style reusable)
- Lambda count after cleanup: 5 of 10 (D2 starter + chunker + validator + lander + dlq_replayer); 5 left for D6 (1 ETL) + D8 (3) + D9 (1 wrapper)
- AWS Bedrock Runtime invocations: ~0 today — D5 has no scripted boto3.bedrock-runtime calls; Cascade IDE authoring is NOT counted against the AWS Bedrock budget. Cumulative still ~2-5 of 50 (Day 1 verify + D3 Lab 2). Cascade IDE usage tracked separately.

```
# Optional verification
git log --oneline module-5-$LEARNER -3

# Cleanup the lab-only test queue + validator + ESM:
ESM_UUID=$(aws lambda list-event-source-mappings \
  --function-name m5-test-validator-$LEARNER \
  --query 'EventSourceMappings[0].UUID' --output text --region ap-south-1)
aws lambda delete-event-source-mapping --uuid $ESM_UUID --region ap-south-1 || true
aws lambda delete-function --function-name m5-test-validator-$LEARNER --region ap-
south-1
aws sqs delete-queue --queue-url $TEST_QUEUE_URL --region ap-south-1
aws sqs delete-queue --queue-url $TEST_DLQ_URL --region ap-south-1

# Lambda count check (target: 5 functions named accelya-*-$LEARNER):
aws lambda list-functions --region ap-south-1 \
  --query "Functions[?starts_with(FunctionName,'accelya-') &&
contains(FunctionName,'$LEARNER')].FunctionName"

# Step Functions state machines (target: 2 — validator-pipeline + dlq-replayer):
aws stepfunctions list-state-machines --region ap-south-1 \
  --query "stateMachines[?contains(name,'$LEARNER')].name"

# Bedrock budget (cross-platform date helper from D3/D4):
PROFILE=apac.anthropic.claude-3-5-sonnet-20241022-v2:0
START=$(python3 -c 'from datetime import datetime,timezone; \

print(datetime.now(timezone.utc).replace(hour=0,minute=0,second=0,microsecond=0).is
oformat().replace("+00:00","Z"))')
END=$(python3 -c 'from datetime import datetime,timezone; \
  print(datetime.now(timezone.utc).isoformat().replace("+00:00","Z"))')
for METRIC in Invocations InputTokenCount OutputTokenCount; do
  aws cloudwatch get-metric-statistics \
```

Trainer note: verify the exact CloudWatch ModelId dimension in console first; AWS Bedrock Runtime calls count ONLY when invoked via direct boto3.bedrock-runtime through the sandbox profile, Cascade IDE usage is tracked separately. Sandbox caps: 50 AWS Bedrock Runtime invocations AND 20,000 tokens. Final Lambda count after cleanup: 5 of 10 — 5 left for D6 (1) + D8 (3) + D9 (1). Do NOT delete validator-pipeline state machine, chunker/validator/lander, dlq-replayer, or DDB — Day 6 reuses them.

What you actually shipped

4 → 5

LAMBDAS (NEW / CUM)

4 new today (chunker, validator, lander, dlq_replayer) + 1 from Day 2 (validator-day2 still live as smoke-test target). Cumulative 5 of 10. 5 left for D6 (+1 ETL) + D8 (+3) + D9 (+1 wrapper).

2

STATE MACHINES

validator-pipeline + dlq-replayer. ASL JSON committed.

1

FULL DLQ CYCLE

Lab-2 poison → DLQ → Lab-3 fix → re-drive → validated/. Documented.

~5

AWS BEDROCK RUNTIME (TODAY)

~0 AWS Bedrock Runtime today (D5 has no scripted boto3.bedrock-runtime calls; Cascade IDE authoring is separate). Cumulative still ~2-5 of 50 (Day 1 verify + D3 Lab 2).

DAY 5 → DAY 6 · WHAT'S COMING TOMORROW

Day 6 — Data Transformation, Cataloguing & Analytics (Glue + Parquet + Athena)

Today you landed JSON files in validated/. They aren't queryable. Tomorrow you turn that into a Parquet-partitioned catalog with Glue Crawler — and immediately query it with Athena. A CATALOG CONTEXT block joins .windsurfrules (→ v3): table name, partition columns (ingest_date + source_system), Parquet vs JSON, schema version.

FROM JSON TO PARQUET TO ATHENA

validated/<learner>//*.jsonl → Glue Crawler → curated/<dataset>/*.parquet → Athena workgroup**

ETL Lambda (1 new Lambda, count → 6) reads validated JSON, writes partitioned Parquet with Snappy compression. Glue Crawler discovers the schema. Athena workgroup with cap queries it.

.WINDSURFRULES v3

CATALOG CONTEXT block joins the v2 file

Cascade now reads catalog metadata (table name, partition columns, Parquet vs JSON, schema version) before scaffolding any read or write to curated/.

SANDBOX BUDGET

Day 6 uses ~1 Lambda + 1 Glue Crawler + 1 Glue Job (≤4 DPU) + Athena workgroup

Cumulative after Day 6: 6 of 10 Lambdas (+1 ETL Lambda); ~7-12 of 50 AWS Bedrock Runtime invocations (D6 typically adds ~5 from DDL refinement + Athena prompt design); Glue 1 of 3 jobs / 4 of 4 DPU / ~2 of 6 DPU-h. Headroom remains for D7+.

Day 6 branch: module-6-<learner>. Inherits .windsurfrules v2 (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT). Bring your validator-queue and DynamoDB table — Day 6 reads from them.

DAY 5 · WRAPPED

Pipeline ships.
ASL is a first-class artefact.

FOUR LAMBDAS

validator + chunker + lander + dlq_replayer. Each single-responsibility. Each Cascade-authored against design doc + 5-section prompt + .windsurfrules v2.

ASL + DLQ

validator-pipeline.asl.json with 3 Lambda Tasks + 2 control states. DLQ replayer with 3 classifiers. One full DLQ→replay→success cycle proven live.

.WINDSURFRULES v2

Two batch blocks added in .windsurfrules v2 (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT). Custom exceptions named. ASL Catch routing as the discipline.

Day 6 starts at 09:00. Bring your validated/ outputs and your DynamoDB table. The catalog enters, and the CATALOG CONTEXT block in .windsurfrules (v3) joins the .windsurfrules v2 you used today (ASL CHUNK CONTEXT + SQS PARTIAL-BATCH CONTEXT).

Accelya Kale · Agentic Development using AWS · Trainer: Jai Singh